

JetRover

A Language-Guided Mobile Manipulator

Closing the perception → planning → action → memory loop on real hardware

[Your Name] · 2026

Hiwonder JetRover · mecanum base + 5-DOF arm · Orbbec RGB-D camera · ROS

Motivation

“I want a coke — can you get me one?”

A useful home / lab robot has to take a vague, natural-language request and carry it out end-to-end — understand it, explore, find the object, and physically fetch it.

[scene photo: robot in the room]

Why it's hard

- **Long-horizon**

“go to A, then B” — chained navigation + manipulation, not one step

- **Partially observable**

objects and obstacles are unknown until the robot actually sees them

- **Real hardware, not simulation**

noisy depth, reflective / transparent objects, limited arm reach

- **One closed loop**

language → navigation → perception → grasp → memory, all tied together

The Platform

- **Mobile base**

4-wheel mecanum — omnidirectional driving

- **Arm**

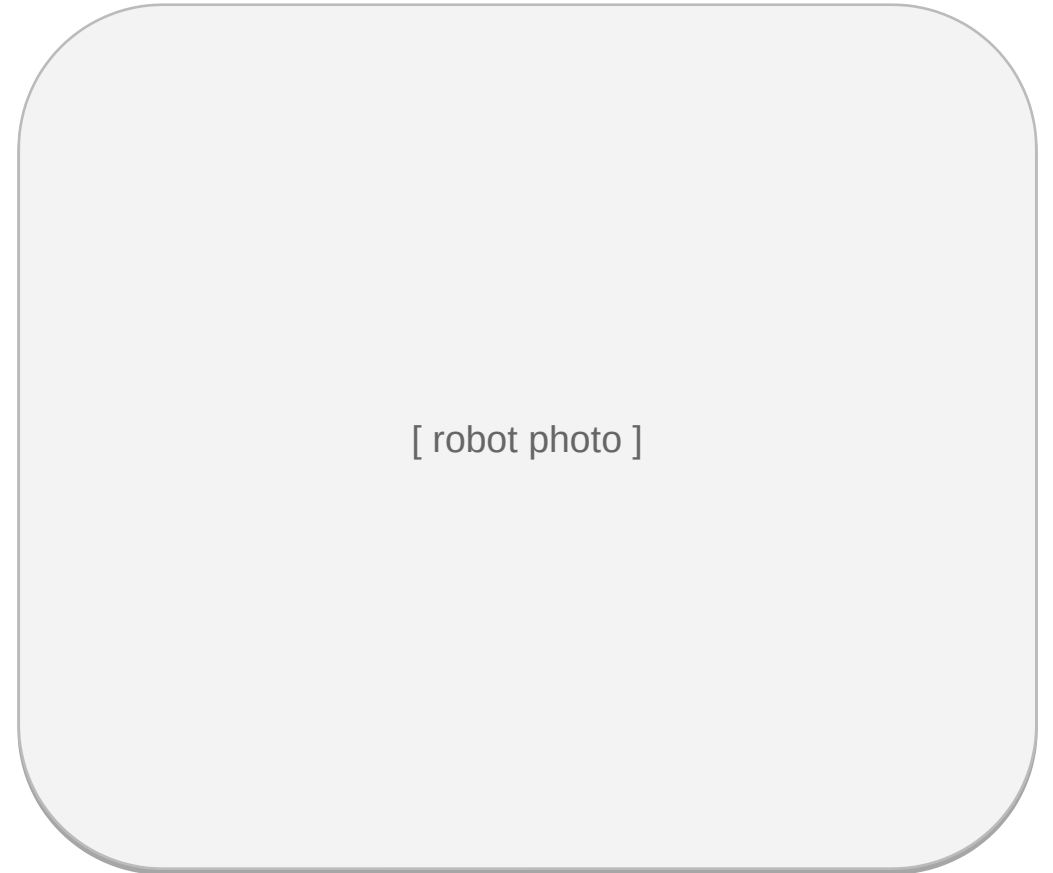
5-DOF manipulator + parallel gripper

- **Sensing**

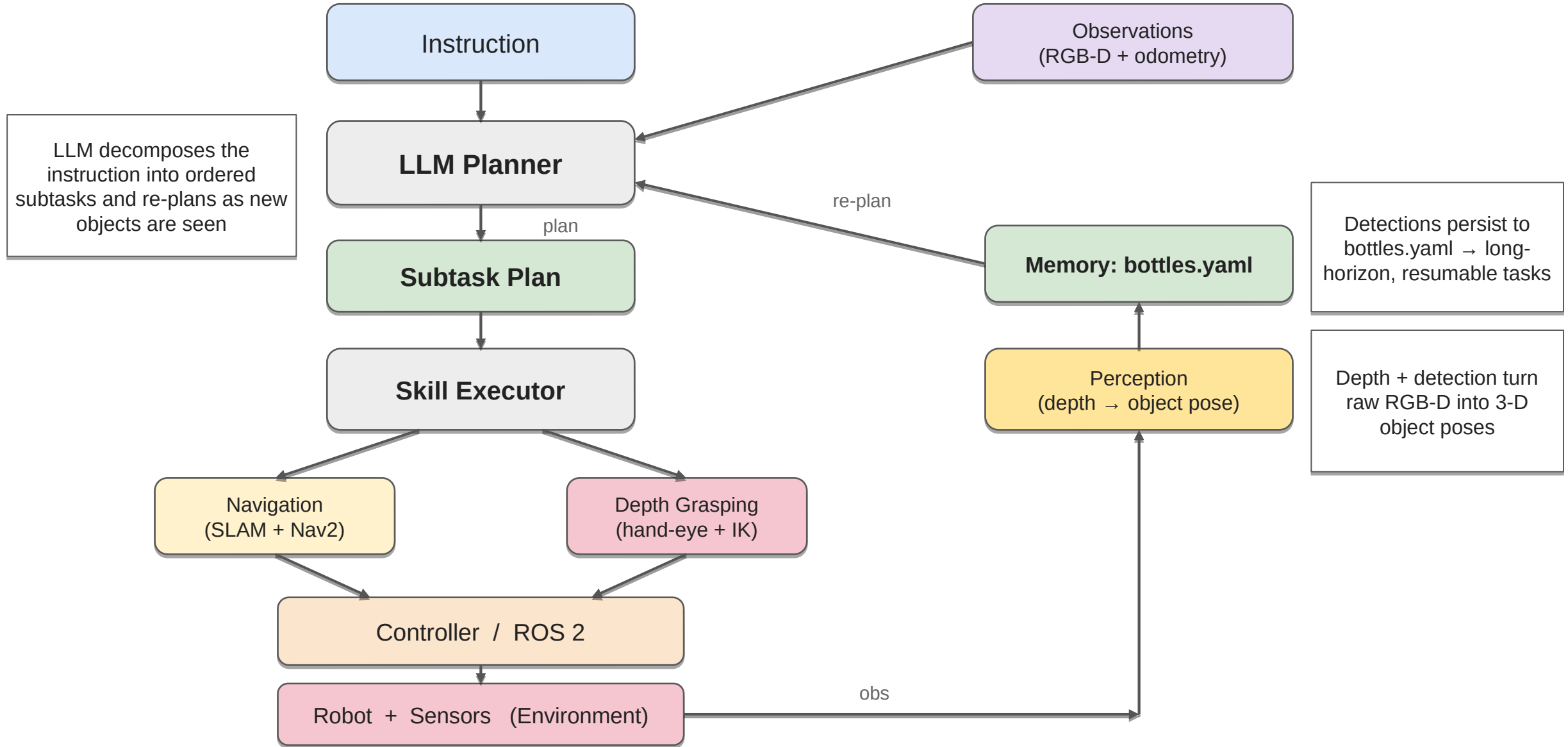
Orbbec DaBai DCW RGB-D depth camera (eye-in-hand)

- **Compute**

onboard Jetson, ROS control stack



System Architecture



Navigation: long-horizon & map-aware

- **SLAM + Nav2**

Hector SLAM builds the map; Nav2 plans and drives to goals

- **Multi-hop goals**

“go to A, then B” verified on hardware — two hops, both reached

- **Reactive to the world**

obstacles that enter the map are planned around

- **Tuned for the real bot**

throttled teleop + path-bias tuning to stay stable, avoid map corruption

[rviz map / trajectory screenshot]

Depth-based Grasping

- **Pipeline**

RGB-D → object detection → hand-eye (eye-in-hand) → IK → approach / descend / close / lift

- **First pick on hardware**

lifted a wrapped can off the table end-to-end

- **Hard problems solved**

in-process FK/IK when the vendor IK service returned no solution; nav + grasp share one control node without contending for the serial bus

- **Open challenge**

reflective / transparent bottles lose depth — robust depth sampling in progress

[grasp photo / depth view]

Memory closes the loop

- **Persist what you see**

every detected object is written to bottles.yaml with its pose

- **Plan over memory**

the LLM filters candidates and orders multi-point navigation to collect them

- **Why it matters**

this is what turns one-shot reactions into long-horizon, resumable tasks

[bottles.yaml → planned route diagram]

Experiments — 5-bottle task

Experiment A — targeted collection

Task: fetch all the Coke bottles

Result: 3 / 3 collected

Experiment B — full patrol

Task: visit and log every bottle

Result: 5 / 5 found

Milestone (2026-07-08): first full perception → planning → action → memory loop running on the real robot — LLM candidate filtering + multi-point navigation + on-disk memory.

Demo

[video / GIF]
Exp A — collect Coke

[video / GIF]
Exp B — patrol

Key Takeaways

- **Full-stack robotics, on real hardware**

ROS, Hector SLAM + Nav2, hand-eye calibration, LLM planning + memory, systems debugging

- **Hardest wins**

nav + grasp coexistence on a single control node; in-process IK; robust depth on tough surfaces

- **Honest next steps**

reliable grasp on reflective bottles; tighter reach calibration

[your email] · [github / demo link]